MACHINE LEARNING

LAB EXPERIMENT / OBSERVATION

Name	Iqbal Ahmed
Register Number	192421320
Course Code	ITA0614
Course Name	Machine Learning

Experiment Question

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Contents: Aim, procedure, algorithm, five example problems, Python code, sample CSV input, sample output, and result.

1. Aim

To implement the Candidate-Elimination algorithm in Python using training examples stored in a CSV file and determine the version space: the set of hypotheses consistent with all the training examples.

2. Basic Concept

Candidate-Elimination maintains two boundaries of the version space:

- **S boundary (Specific boundary):** the set of maximally specific hypotheses consistent with the observed data.
- **G boundary (General boundary):** the set of maximally general hypotheses consistent with the observed data.

Initially, S is the most specific hypothesis and G is the most general hypothesis. For a positive example, S is minimally generalised and inconsistent members of G are removed. For a negative example, G is minimally specialised and inconsistent members of S are removed. The hypotheses between S and G represent the version space.

Symbols

- **Ø** — no attribute value accepted yet; used in the initial specific hypothesis.
- **?** — any value is accepted for that attribute.
- **Version space** — all hypotheses that correctly classify every training example seen so far.

3. Procedure

1. Create a CSV file with attribute columns and a final target column (Yes/No).
2. Read the CSV file using Python's csv module.
3. Initialise S to the most specific hypothesis and G to the most general hypothesis.
4. Process each training example in order.
5. For a positive example, remove inconsistent hypotheses from G and minimally generalise S to cover the example.
6. For a negative example, remove inconsistent hypotheses from S and minimally specialise G to exclude the example.
7. Remove redundant hypotheses from both boundaries.
8. Print S and G; these boundaries describe the version space under the assumed finite attribute domains.

4. Algorithm / Pseudocode

```
Initialise:
    S = {<Ø, Ø, ..., Ø>}
    G = {<?, ?, ..., ?>}

For each example x:
    If x is positive:
        Remove from G every hypothesis that does not cover x
        Generalise members of S minimally to cover x
        Remove hypotheses in S that are more general than needed
    Else (x is negative):
        Remove from S every hypothesis that covers x
        Specialise members of G minimally to exclude x
        Remove hypotheses in G that are less general than needed

Return S and G as the version-space boundaries
```

The program uses a finite set of values observed in the CSV file to generate minimal specialisations for categorical attributes. Candidate-Elimination assumes noise-free, correctly labelled examples and a suitable hypothesis space.

5. Five Example Problems

Example 1 — EnjoySport (step-by-step)

Attributes: Sky, AirTemp, Humidity. Target: EnjoySport. For clarity, this compact dataset demonstrates the boundary updates.

Sky	AirTemp	Humidity	EnjoySport
Sunny	Warm	Normal	Yes
Sunny	Warm	High	Yes
Rainy	Cold	High	No

After Example 1 (positive): S = {■Sunny, Warm, Normal■}; G = {■?, ?, ?■}.

After Example 2 (positive): S = {■Sunny, Warm, ?■}; G = {■?, ?, ?■}.

After Example 3 (negative): remove G hypotheses that cover ■Rainy, Cold, High■. Minimal specialisations consistent with the positive examples are ■Sunny, ?, ?■ and ■?, Warm, ?■. So G = {■Sunny, ?, ?■, ■?, Warm, ?■}; S remains {■Sunny, Warm, ?■}.

Final boundaries: S = {■Sunny, Warm, ?■}; G = {■Sunny, ?, ?■, ■?, Warm, ?■}.

Interpretation: all hypotheses between these S and G boundaries are consistent with the three examples in the chosen hypothesis space.

Example 2 — Student pass prediction

Training examples:

Attendance	StudyHours	Class
High	Many	Yes
High	Few	Yes
Low	Many	No

Final S boundary: {■High, ?■}

Final G boundary: {■High, ?■, ■?, Few■}

Explanation: The positive examples force StudyHours to become ?. The negative example ■Low, Many■ is excluded by specialisations that retain consistency with the positives.

Example 3 — Outdoor activity

Training examples:

Weather	Wind	Class
Clear	Low	Yes
Clear	High	Yes
Cloudy	Low	No

Final S boundary: {■Clear, ?■}

Final G boundary: {■Clear, ?■}

Explanation: Both positive examples require Weather = Clear and allow any Wind. The negative example is excluded by the boundary hypothesis ■Clear, ?■.

Example 4 — Loan pre-screening demonstration

Training examples:

Income	Credit	Class
High	Good	Yes
Medium	Good	Yes
Low	Poor	No

Final S boundary: {■?, Good■}

Final G boundary: {■?, Good■}

Explanation: Positive samples vary in Income but share Credit = Good. The negative sample has Credit = Poor and is excluded. This is a toy algorithm example, not a real lending rule.

Example 5 — Water sample alert

Training examples:

Turbidity	Conductivity	Class
High	High	Yes
High	Low	Yes
Low	High	No

Final S boundary: {■High, ?■}

Final G boundary: {■High, ?■}

Explanation: Positive samples share High turbidity while conductivity varies. The negative sample has Low turbidity and is excluded. This example demonstrates the algorithm only; it is not a water-safety decision rule.

6. Sample CSV Input

Save the following data in a file named **enjoysport.csv**. The final column must be named EnjoySport and contain Yes or No.

```
Sky,AirTemp,Humidity,EnjoySport
Sunny,Warm,Normal,Yes
Sunny,Warm,High,Yes
Rainy,Cold,High,No
Sunny,Warm,High,Yes
```

7. Python Program

Save the program as **candidate_elimination.py** in the same folder as the CSV file. This implementation uses the observed categorical values as each attribute's finite domain.

```
import csv
from itertools import product

def covers(hypothesis, example):
    return all(h == "?" or h == x for h, x in zip(hypothesis, example))

def more_general_or_equal(h1, h2):
    # True when h1 is at least as general as h2
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def minimal_specialisations(g, negative, domains, s_boundary):
    results = set()
    for i, value in enumerate(g):
        if value == "?":
            for domain_value in domains[i]:
                if domain_value != negative[i]:
                    candidate = list(g)
                    candidate[i] = domain_value
                    # Keep only candidates that still cover some S boundary
                    if any(covers(candidate, s) for s in s_boundary):
                        results.add(tuple(candidate))
    return results

def candidate_elimination(csv_file):
    with open(csv_file, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        rows = list(reader)
        headers = reader.fieldnames

    if not rows or not headers or len(headers) < 2:
        raise ValueError("CSV must contain attributes and a target column.")

    target = headers[-1]
    attribute_names = headers[:-1]
    X = [[row[h].strip() for h in attribute_names] for row in rows]
    y = [row[target].strip() for row in rows]

    domains = [sorted({x[i] for x in X}) for i in range(len(attribute_names))]
    n = len(attribute_names)
    S = {tuple(["Ø"] * n)}
    G = {tuple(["?"] * n)}

    for x, label in zip(X, y):
        if label.lower() == "yes":
            G = {g for g in G if covers(g, x)}
            new_S = set()
            for s in S:
                if all(v == "Ø" for v in s):
                    new_S.add(tuple(x))
                elif covers(s, x):
                    new_S.add(s)
                else:
                    new_S.add(tuple(
                        old if old == val else "?"
                        for old, val in zip(s, x)
                    ))
            S = {
                s for s in new_S
                if not any(s != other and more_general_or_equal(s, other)
                           for other in new_S)
            }
        elif label.lower() == "no":
```

```
            S = {s for s in S if not covers(s, x)}
            new_G = set()
            for g in G:
                if not covers(g, x):
                    new_G.add(g)
                else:
                    new_G.update(minimal_specialisations(g, x, domains, S))
            G = {
                g for g in new_G
                if not any(g != other and more_general_or_equal(other, g)
                           for other in new_G)
            }
        else:
            raise ValueError("Target labels must be Yes or No.")

    fmt = lambda boundary: sorted(["<" + ", ".join(h) + ">" for h in boundary])
    print("Attributes:", attribute_names)
    print("S boundary:", fmt(S))
    print("G boundary:", fmt(G))

    if not S or not G:
        print("Version space is empty: examples may conflict with the hypothesis space.")
    return S, G

if __name__ == "__main__":
    candidate_elimination("enjoysport.csv")
```

8. Sample Output

```
Attributes: ['Sky', 'AirTemp', 'Humidity']
S boundary: ['<Sunny, Warm, ?>']
G boundary: ['<?, Warm, ?>', '<Sunny, ?, ?>']
```

The displayed boundaries are the output for the CSV rows shown above. Depending on the attribute domains and hypothesis-space implementation, boundary ordering may differ; the hypothesis set is what matters.

9. Result

Thus, the Candidate-Elimination algorithm was implemented in Python to read training samples from a CSV file and produce the specific boundary S and general boundary G. Together, these boundaries describe the hypotheses consistent with the observed training data under the chosen hypothesis space.

Precautions / Limitations

- Training data should be correctly labelled and free from contradictory noise.
- The algorithm assumes categorical attributes with known finite domains in this implementation.
- If examples contradict every hypothesis in the selected space, the version space becomes empty.
- Candidate-Elimination can be sensitive to noisy examples and is mainly useful for demonstrating concept learning.